Task 2: Reusable Feature Pipeline

Documentation | Pipeline version **features-1.0.0** | Prepared by Katarukonda Ashok (GM00154) | October 2026

1. What this task is, in plain words

Raw HR data is messy, and every model needs it cleaned the same way. Think of the pipeline as a **recipe card** written once and reused by every model: it repairs bad values, builds new features, fills gaps, scales numbers and encodes categories. Because the recipe is saved and versioned, the live service will prepare data exactly as training did, and model scores can be compared fairly.

One rule governs the whole task: the pipeline may only *learn* (medians, caps, category lists, department pay levels) from the **training rows**. Learning from all rows before splitting lets test information leak into training.

2. What the pipeline does, step by step

Step	Class (file src/features/)	What it does	Learns from train?
1 clean	RowCleaner (transformers.py)	Fixes city spellings, negative tenure, attendance above 1, wfh_share outside 0-1, negative experience, experience shorter than tenure	No (fixed rules)
2 fix_salary	SalaryTypoFixer	Finds pay about 10x too high by comparing it with the role's usual pay band, then divides by 10	Yes: pay band of each role
3 engineer	FeatureEngineer	Adds flags (never_promoted, no_raise_yet, not_reviewed_yet, engagement_missing) and new features; fills not-applicable gaps	Yes: department median pay
4 encode	ColumnTransformer (pipeline.py)	Numbers: cap at 1st/99th percentile, median fill, scale. Flags: pass through. Categories: fill Unknown, one-hot. Everything else is dropped	Yes: caps, medians, scales, categories

Features requested by the task

Requested	Column(s) in the output
Attendance rate	attendance_rate_3m, attendance_gap (1 minus rate)
Late count	late_arrivals_3m
Average overtime	avg_overtime_hours_per_month_3m, overtime_per_year_of_tenure
Tenure	tenure_months, tenure_years
Salary versus department median	pay_vs_dept_median (pay divided by the department median learned on train), log_pay
Months since last revision	months_since_last_raise (never-raised people are flagged by no_raise_yet)

3. Audit decisions turned into code

Task 1 decision	Where it lives
D1 duplicates	data.drop_duplicates, before the split
D3 cap skewed values	Winsorizer
D6 salary typos	SalaryTypoFixer
D7-D11, D20 impossible values	RowCleaner
M1-M4 'not applicable' gaps	FeatureEngineer flags plus sensible fills
M5-M6 small random gaps	median fill / category Unknown
D15-D16 leakage and identifiers	never selected by the ColumnTransformer (remainder='drop')
D17 fit on train only	fit_pipeline(train) and cloning inside cross-validation

4. Results

Check	Result
Rows	2,389 raw, 2,383 after removing duplicates; split into 1,906 train and 477 test (leaver rate 14.6% and 14.7%: stratified)
Output columns	74 = 29 numeric + 4 flags + 41 one-hot categories
Missing values after transform	0
Leakage / identifier columns in output	0
Salary typos fixed in training rows	2 (the third typo row fell in the test part and is fixed there using train-learned pay bands)
Save, load, transform	Loaded pipeline gives identical output: True

Check	Result
Version in metadata	features-1.0.0
Automated tests	15 passed in 2.74s
One-command run (Docker/CLI)	Works

Why train-only fitting matters (example): department median pay computed from train and from all rows differ slightly. If the all-rows version were used, the test rows would influence the features the model trains on. The notebook (Step 7) prints both side by side.

5. How to run it

The files live in `src/features/` (config, data, transformers, pipeline, cli). From the project root:

```
python -m src.features.cli run --input data/raw/02_CortexHR_AI_ML_Dataset.xlsx --output-dir artifacts
python -m src.features.cli transform --model artifacts/features-1.0.0.joblib --input new_rows.csv --output features.csv
```

The first command removes duplicates, splits, fits on train, saves `features-1.0.0.joblib` plus a metadata file, and writes `X_train.csv` and the locked test files. Docker version:

```
docker build -t aura-attrition .
docker run --rm -v "$PWD:/app" aura-attrition python -m src.features.cli run --output-dir artifacts
```

Use it in Python:

```
from src.features.pipeline import fit_pipeline, save_pipeline, load_pipeline, transform_dataframe
pipe = fit_pipeline(train_df)                       # train rows only
save_pipeline(pipe, train_df, 'artifacts')
pipe, meta = load_pipeline('artifacts/features-1.0.0.joblib')
X = transform_dataframe(pipe, new_rows_df)
```

6. Completion criteria

Completion criterion	Status	Evidence
Pipeline fits, saves, loads and transforms	Done	Step 8 of the notebook; identical output after reload
Version number recorded	Done	features-1.0.0 inside the saved file and the .meta.json
Tests pass	Done	tests/test_features.py: output shape, no missing values, same input same output, save/load, no leakage, train-only fitting
Runs inside Docker / from the command line	Done*	CLI verified; *Dockerfile provided (see README to build)

Next: Task 3 plugs this pipeline in front of each baseline model and refits it inside every cross-validation fold, so even the medians and caps are learned from training folds only.